

## 06 - Plan, Drift, Projection, Roadmap, Scenarios, and History

### Feature objective

Make the plan the product's signature experience: a deterministic financial projection plus an actionable roadmap, with explicit baseline versioning, safe what-if scenarios, and user-confirmed plan drift updates.

### Dependencies

- Accounts/Financial State
- Transactions
- Goals
- Financial Health
- backend plan/scenario engine

### In scope

- current baseline plan
- plan status
- projection preview and full view
- roadmap preview and full view
- recommendations
- risks
- regenerate plan
- data freshness gate/warnings
- scenario builder
- scenario comparison
- plan drift review
- 7-day lightweight drift signal
- 30-day deeper review result
- plan history/version detail
- restore old version as a new version

### Out of scope

- silent baseline mutation
- mobile-side financial modeling
- arbitrary AI-generated numbers
- unlimited advanced quant assumptions in MVP

### Routes

```
(tabs)/plan.tsx
plan/projection.tsx
plan/roadmap.tsx
plan/history.tsx
plan/version/[id].tsx
plan/review/[id].tsx
scenario/new.tsx
scenario/[id].tsx
```

### Feature folders

```
features/plan/
|-- screens/
|-- components/
|-- hooks/
|-- services/
```

```
|-- utils/
`-- types.ts

features/scenarios/
|-- screens/
|-- components/
|-- forms/
|-- hooks/
|-- services/
`-- schemas/
```

## Plan root screen

High-level hierarchy:

1. current plan version + last updated
2. overall status
3. next milestone
4. roadmap preview
5. projection preview
6. top recommendations
7. risks
8. actions

Primary actions:

- Run scenario
- Regenerate plan
- Export report later/P1

Do not render every plan chart on the root screen.

## Projection

Standard mobile projection view:

- Net Worth
- Cash
- Debt
- Investments
- Goals

Allow scenario bands:

- Conservative
- Expected
- Optimistic

Victory Native is appropriate for standard charts. Use Skia directly only when a custom interaction cannot be expressed cleanly.

Every chart requires a text summary, e.g.:

Expected net worth reaches approximately INR 1.21 Cr by 2041 under the current baseline.

## Roadmap

Roadmap answers “what do I do and when?”

Each item should expose:

- time period/date
- action
- amount/target when relevant

- dependency
- current status
- why it matters

Example:

Aug–Nov 2026  
Build emergency fund  
INR 3,000/month

Mar 2027  
Bike EMI completes  
Frees INR 4,200/month

Apr 2027  
Increase SIP  
INR 10,000 -> INR 14,000

## Plan regeneration

Before serious regeneration:

1. fetch financial freshness summary
2. if major account is 7-30 days stale -> warn
3. if over 30 days stale -> strong warning
4. allow Update Info or Continue Anyway
5. start server plan job
6. stream progress/status if supported
7. never replace current plan until generation succeeds

If generation fails, existing plan stays active.

## Baseline rule

Exactly one current baseline plan.

Scenarios are forks.

A scenario may be saved, deleted, duplicated, or applied.

Apply means:

- show explicit confirmation
- backend creates a new baseline plan version
- old baseline remains in history

## Scenario variables - MVP

- income
- recurring expenses
- SIP/investment contribution
- loan prepayment
- goal amount
- goal date

Do not expose every internal assumption initially.

## Scenario result

Always compare to baseline.

At minimum:

- monthly surplus
- goal completion dates
- projected net worth
- debt timeline
- feasibility/risk flag

Example:

| Baseline    | Scenario              |
|-------------|-----------------------|
| INR 18.1k   | INR 30.1k surplus     |
| May 2028    | Sep 2027 home goal    |
| INR 1.21 Cr | INR 1.48 Cr net worth |

## Drift model

### Lightweight check - approximately every 7 days

Detect meaningful deviations such as:

- spending above baseline
- income change
- new recurring obligation
- missed expected contribution
- goal progress change

Do not notify if changes are noise.

### Deep review - approximately every 30 days

Backend recomputes:

- projections
- goal feasibility
- risk state
- next actions
- research opportunities only when materially useful

## Drift review screen

Show three sections:

1. What changed
2. What it affects
3. Proposed adjustment

Actions:

- Accept new baseline
- Keep current plan
- Review transactions

Never silently rewrite baseline.

## Plan history

List versions by date and reason.

Version detail should show meaningful diffs:

- income
- expenses
- goal assumptions
- loan state
- recommendations

- projected outcomes

Restore old version:

- create a new version based on old assumptions/state
- do not delete later history

## API requirements

Conceptual:

```
GET    /plans/current
POST   /plans/generate
GET    /plans/generations/:jobId
GET    /plans/history
GET    /plans/:id
POST   /plans/:id/restore
```

```
POST   /scenarios
GET    /scenarios/:id
PATCH /scenarios/:id
POST   /scenarios/:id/apply
DELETE /scenarios/:id
```

```
GET    /plan-drift/current
POST   /plan-drift/:id/accept
POST   /plan-drift/:id/keep
```

## Query hooks

```
useCurrentPlan
usePlanVersion
usePlanHistory
useGeneratePlan
usePlanGenerationStatus
useCreateScenario
useScenario
useApplyScenario
useCurrentDrift
useResolveDrift
```

## Offline behavior

- cached current plan/projection can render offline with freshness label
- creating/applying a scenario requires internet
- plan regeneration requires internet
- drift resolution requires internet

## Step-by-step implementation

1. Build plan root with mock/SDK data.
2. Build projection chart + text summary.
3. Build roadmap timeline.
4. Add recommendations/risks sections.
5. Add freshness warnings and regeneration flow.
6. Build scenario form.
7. Build baseline comparison result.
8. Add save/apply/discard scenario.
9. Build drift review.

10. Build plan history/version detail.
11. Add restore-as-new-version.
12. Add cached/offline read state.

## Acceptance criteria

- current plan remains usable if regeneration fails
- projection has accessible text summary
- scenario never mutates baseline before confirmation
- applying scenario creates/loads a new baseline version
- drift review clearly shows cause and consequence
- user can keep old plan after drift
- user can restore historical assumptions without deleting history
- stale critical data warning appears before serious regeneration

## Tests

- plan state rendering
- stale data gate
- scenario form validation
- baseline vs scenario display
- apply confirmation
- failed generation preserves old plan
- drift accept/keep flows
- history restore action